Configurable Approval Workflow Engine

Comprehensive Architectural Blueprint, Domain Model, Request Flow, Exception Handling & API Reference

Author: Senior Java Backend Engineering Team | **Stack:** Java 21, Spring Boot 3.3.5, Spring Security (JWT), Spring Data JPA, H2, JaCoCo (96% Coverage) | **Target Use Cases:** Employee Leave Requests, Expense Claims, Purchase Orders

1. Executive Summary & Core Objective

The **Configurable Approval Workflow Engine** is an enterprise backend service built to automate multi-stage approval sequences without hardcoding workflow routes or business rules into the source code.

> **CRITICAL ARCHITECTURAL RULE: Zero Hardcoded Workflows**
>
> The engine contains **no if-else or switch-case statements** based on request types (`LEAVE` , `EXPENSE`), role names, or step sequences. All approval steps, sequences, and role authorities are loaded dynamically from the database. Adding a new request type or altering an existing sequence requires only inserting or updating database records.

Key Architectural Guarantees

- **N-Step Workflow Agnostic:** Works seamlessly for 1-step (`QUICK_REQUEST`), 2-step (`LEAVE` , `EXPENSE`), 3-step (`PURCHASE_ORDER`), or any N-step sequence.
- **Workflow Versioning / Snapshots:** Requests capture the active `workflowVersion` at submission time. Mid-flight requests are immune to database configuration changes.
- **Concurrency Control:** Implements JPA `@Version` optimistic locking to guarantee that concurrent approvals cannot duplicate step advancements.
- **Immutable Append-Only Auditing:** Every action generates a tamper-proof audit record in `approval_history` . No updates or deletions are exposed.
- **Stateless JWT Security:** Signed HMAC-SHA256 tokens authenticate all business endpoints. Requesters are strictly prohibited from approving their own requests.

2. Layered Architecture & Package Structure

The codebase strictly follows a clean layered architecture ensuring loose coupling and separation of concerns:

Package	Key Classes	Responsibility

`com.example.workflow.controller`	`RequestController`
	`AdminWorkflowController`
	`AuthenticationController`
		Thin REST controllers handling HTTP requests, input validation annotations (`@Valid`), and delegating directly to services.

`com.example.workflow.service`	`WorkflowService`
	`WorkflowConfigurationService`
	`RequestService`
	`ApprovalHistoryService`
		Core domain logic. `WorkflowConfigurationService` queries dynamic DB steps. `WorkflowService` executes lifecycle state transitions.

`com.example.workflow.entity`	`Request`
	`ApprovalStep`
	`ApprovalHistory`
	`User`
		JPA entities mapping domain models to database tables with primary keys, indexes, foreign keys, and optimistic locking tokens.

`com.example.workflow.security`	`JwtService`
	`JwtAuthenticationFilter`
	`CustomUserDetailsService`
	`SecurityUtils`
		Cryptographic JWT token generation, stateless Bearer token extraction filter, and principal context resolution.

`com.example.workflow.exception`	`GlobalExceptionHandler`
	`WorkflowTransitionException`
	`UnauthorizedActionException`
		Centralized `@RestControllerAdvice` mapping application exceptions into consistent JSON error responses.

`com.example.workflow.config`	`SecurityConfig`
	`OpenApiConfig`
	`DatabaseSeeder`
		Spring Security filter chain, Swagger OpenAPI 3.0 documentation, and development database bootstrapping.

3. Database Design & Entity Relationships

The database schema consists of four relational tables designed for transactional integrity and auditability:

Table Name	Primary Key & Constraints	Key Columns	Purpose
`users`	`id` (PK, Identity) `username` (Unique, Index)	`username`, `password_hash`, `role` (`REQUESTER`, `APPROVER`, `ADMIN`), `enabled`, `created_at`	Stores system users with BCrypt password hashes. Plaintext passwords are never stored.
`approval_steps`	`id` (PK, Identity) Unique: `(request_type, workflow_version, step_order)`	`request_type`, `step_order`, `role`, `workflow_version`, `active`	Defines dynamic workflow sequences in the database. Loaded on-demand by the engine.
`requests`	`id` (PK, Identity) Indexed: `created_by`, `status`, `type`	`type`, `status` (`PENDING`, `APPROVED`, `REJECTED`), `created_by`, `current_step_order`, `workflow_version`, `payload` (CLOB), `version` (Optimistic Lock)	Represents workflow requests. Tracks active step, payload JSON, and optimistic locking token.
`approval_history`	`id` (PK, Identity) Indexed: `request_id`, `action_at`	`request_id`, `action`, `action_by`, `action_at`, `step_order`, `remarks`	Immutable, append-only audit trail logging every creation, approval, rejection, and override.

Sample Database Configurations Seeded at Boot:

- `LEAVE` **(2 Steps):** Step 1: `APPROVER` (Manager) → Step 2: `ADMIN` (HR Executive)
- `EXPENSE` **(2 Steps):** Step 1: `APPROVER` (Supervisor) → Step 2: `ADMIN` (Finance Director)
- `PURCHASE_ORDER` **(3 Steps):** Step 1: `APPROVER` → Step 2: `APPROVER` → Step 3: `ADMIN`
- `QUICK_REQUEST` **(1 Step):** Step 1: `APPROVER` (Concludes immediately upon single approval)

4. End-to-End Request Flow

The following step-by-step breakdown illustrates how a request flows through the entire system architecture:

1. **Authentication & Security Interception (`POST /auth/login`)**
   Client submits credentials. Spring Security verifies the BCrypt hash. `JwtService` issues a signed HMAC-SHA256 token containing the username and authority claim (e.g., `ROLE_REQUESTER`). On all subsequent calls, `JwtAuthenticationFilter` validates the Bearer token and establishes the `SecurityContext`.

2 **Request Submission (`POST /requests`)**
The client submits `{"type": "LEAVE", "payload": {"days": 3}}`. `SecurityUtils` extracts the creator identity directly from the authenticated principal (never trusting client-supplied author headers). `WorkflowConfigurationService` queries the database for active steps for `LEAVE`. Request is saved with status `PENDING`, `currentStepOrder = 1`, and an immutable `REQUEST_CREATED` history entry is saved in the same transaction.

3 **Step Approval & Sequence Advancement (`POST /requests/{id}/approve`)**
Approver submits approval. The engine enforces domain rules:

1. Status must be `PENDING` (otherwise throws `WorkflowTransitionException` → 409 Conflict).
2. Self-approval check: `request.createdBy != approverUsername` (otherwise throws `UnauthorizedActionException` → 403 Forbidden).
3. Role check: Approver's role must match the database-configured role for `currentStepOrder` (otherwise throws 403 Forbidden).

Engine writes an `APPROVED` record to history. It then queries the DB for `currentStepOrder + 1`:

- If a next step exists: Request advances to `currentStepOrder = 2`.
- If no next step exists: Request transitions to terminal status **`APPROVED`** and sets `currentStepOrder = null`.

4 **Workflow Rejection (`POST /requests/{id}/reject`)**
An authorized approver rejects the request. The engine validates authorization, appends a `REJECTED` audit event with mandatory/optional remarks, transitions status to `REJECTED`, and sets `currentStepOrder = null`. Workflow is permanently terminated.

5 **Emergency Administrator Override (`POST /admin/requests/{id}/override`)**
An `ADMIN` user can force-override any request to `APPROVED` or `REJECTED` at any time. The engine verifies the user is an admin, checks that the target status is different from current status (idempotency guard), updates status, and appends `ADMIN_OVERRIDE_APPROVED` or `ADMIN_OVERRIDE_REJECTED` to history without removing past records.

5. Centralized Exception Handling & HTTP Status Code Mapping

Exception handling is implemented in two tiers: at the **Security Filter Layer** and via a centralized `@RestControllerAdvice` in `GlobalExceptionHandler.java`.

Exception Class	HTTP Code	Error Title	Trigger Condition
`MethodArgumentNotValidException`	400 Bad Request	Validation Failed	Bean validation fails (e.g. blank username, blank password, empty request type).
`InvalidWorkflowException`	400 Bad Request	Invalid Workflow Configuration	Requested type has no active steps configured in database, or step sequence has gaps.
`BadCredentialsException`	401 Unauthorized	Authentication Failed	Invalid username or password supplied during login.
`AuthenticationEntryPoint`	401 Unauthorized	Unauthorized	Client calls protected business endpoint without a valid Bearer JWT.
`UnauthorizedActionException`	403 Forbidden	Forbidden Action	Requester attempts self-approval; or non-admin attempts registration for `ADMIN` role.
`AccessDeniedException`	403 Forbidden	Access Denied	User role does not have authorization for endpoint or attempting to view another user's request.
`ResourceNotFoundException`	404 Not Found	Resource Not Found	Request ID, User, or Step cannot be found in the database.
`WorkflowTransitionException`	409 Conflict	Invalid State Transition	Attempting to approve or reject a request that is already approved, rejected, or completed.
`UserAlreadyExistsException`	409 Conflict	User Conflict	Attempting to register with a username that already exists in the database.
`OptimisticLockingFailureException`	409 Conflict	Concurrent Modification Conflict	Two concurrent threads attempt to advance the exact same request simultaneously.
`Exception` (General)	500 Server Error	Internal Server Error	Unhandled runtime crashes. Logged via SLF4J internally; stack traces never leak to client.

Standardized JSON Error Payload Format:

```
{
  "timestamp": "2026-10-28T14:20:00Z",
  "status": 403,
  "error": "Forbidden Action",
  "message": "A requester cannot approve their own request.",
  "path": "/requests/1/approve"
}
```

6. REST API Endpoint Specifications

Method	Route	Authorized Roles	Description	Success Code
`POST`	`/auth/login`	Public	Validates credentials, returns signed JWT token with claims.	200 OK
`POST`	`/auth/register`	Public	Registers new user account (`REQUESTER` or `APPROVER` only).	201 Created
`POST`	`/requests`	`REQUESTER`	Initializes workflow request at step 1; saves audit record.	201 Created
`GET`	`/requests/{id}`	`REQUESTER` , `APPROVER` , `ADMIN`	Retrieves metadata, payload, and dynamic current step role (privacy restricted).	200 OK
`POST`	`/requests/{id}/approve`	`APPROVER` , `ADMIN`	Approves active step; advances sequence or completes workflow.	200 OK
`POST`	`/requests/{id}/reject`	`APPROVER` , `ADMIN`	Rejects request and terminates workflow.	200 OK
`GET`	`/requests/history/{id}`	`REQUESTER` , `ADMIN`	Returns complete immutable chronological audit trail.	200 OK
`POST`	`/admin/requests/{id}/override`	`ADMIN`	Emergency override to APPROVED or REJECTED with mandatory remarks.	200 OK

7. Automated Testing & Code Coverage Report

The project includes an exhaustive automated test suite verified via Maven and JaCoCo:

Coverage Metric	Required Minimum	Actual Result Achieved	Verification Status
Line Coverage	80.00%	**96.00%** (839 / 874 lines)	PASSED
Instruction Coverage	80.00%	**94.75%** (3,391 / 3,579 instructions)	PASSED
Total Automated Tests	—	**56 Tests** (0 Failures, 0 Errors, 0 Skipped)	PASSED

Test Categories Included in Suite:

- **Security & JWT Tests:** Token expiration, invalid signatures, privilege escalation prevention, unauthenticated 401, role forbidden 403.
- **Full Workflow Tests:** Single-step (`QUICK_REQUEST`), 2-step (`LEAVE` , `EXPENSE`), and 3-step (`PURCHASE_ORDER`).
- **Authorization & Privacy Tests:** Requesters cannot approve their own requests; Requesters cannot view another user's requests or audit history.
- **Concurrency & Optimistic Locking Tests:** Multi-threaded concurrent approval tests confirming that duplicate step advancement is prevented and throws 409 Conflict.
- **Transaction Rollback Tests:** Injected audit persistence failures verifying complete transactional rollback without leaving dirty states.

8. Deployment & Execution Instructions

Local Development Startup

```
# 1. Start application using Maven Wrapper
.\mvnw.cmd spring-boot:run

# 2. Access Interactive Swagger UI Documentation
http://localhost:8080/swagger-ui/index.html

# 3. Access H2 In-Memory Database Console
http://localhost:8080/h2-console
JDBC URL: jdbc:h2:mem:workflowdb | Username: sa | Password: password
```

Pre-Seeded Development Credentials

Username	Password	Role	Primary Testing Role
`requester1`	`password123`	`REQUESTER`	Creating workflow requests, viewing own history
`requester2`	`password123`	`REQUESTER`	Testing ownership privacy & multi-user isolation
`approver1`	`password123`	`APPROVER`	Normal approval / rejection of step 1
`approver2`	`password123`	`APPROVER`	Intermediate approvals in multi-stage workflows
`admin`	`admin123`	`ADMIN`	Final approval stage & emergency administrator overrides

Docker & Cloud Deployment (Render)

The project includes a multi-stage `Dockerfile` and `render.yaml`. The configuration dynamically binds to Render's `$PORT` environment variable via `server.port=${PORT:8080}` with memory limit optimizations (`-Xmx384m`) for free-tier cloud containers.